

BLOQUE II • Conversores de Código

## Memoria del Proyecto

### Bloque II (Fase I)

Diseño, implementación y verificación en VHDL

bcd\_to\_bin ↔ bin\_to\_bcd

(Versión 1.0 — Mayo 2026)

📁 **Fase:** Bloque II (Fase I)

📅 **Fecha:** 6 Mayo de 2026



#### Miembros del Grupo

|                       |                                    |
|-----------------------|------------------------------------|
| Lucía Llana Carmona   | lucia.llana@alumnos.upm.es         |
| María Moya Marcilla   | maria.moya.marcilla@alumnos.upm.es |
| Pedro Gil Bolaños     | pedro.gilb@alumnos.upm.es          |
| Pedro Osma Moya       | p.osma@alumnos.upm.es              |
| Eduardo Muñoz de Maya | eduardo.mdemaya@alumnos.upm.es     |

# Índice

|                                                        |          |
|--------------------------------------------------------|----------|
| <b>1. Introducción y Objetivos</b>                     | <b>2</b> |
| 1.1. Arquitectura jerárquica del sistema . . . . .     | 2        |
| <b>2. Conversor BCD a Binario (bcd_to_bin)</b>         | <b>2</b> |
| 2.1. Especificación . . . . .                          | 2        |
| 2.2. Implementación mediante desplazamientos . . . . . | 2        |
| 2.3. Verificación . . . . .                            | 3        |
| <b>3. Conversor Binario a BCD (bin_to_bcd)</b>         | <b>3</b> |
| 3.1. Especificación . . . . .                          | 3        |
| 3.2. Algoritmo: Double Dabble . . . . .                | 3        |
| 3.3. Máquina de Estados (FSM) . . . . .                | 4        |
| 3.4. Verificación . . . . .                            | 5        |
| <b>4. Conclusiones</b>                                 | <b>5</b> |

## 1. Introducción y Objetivos

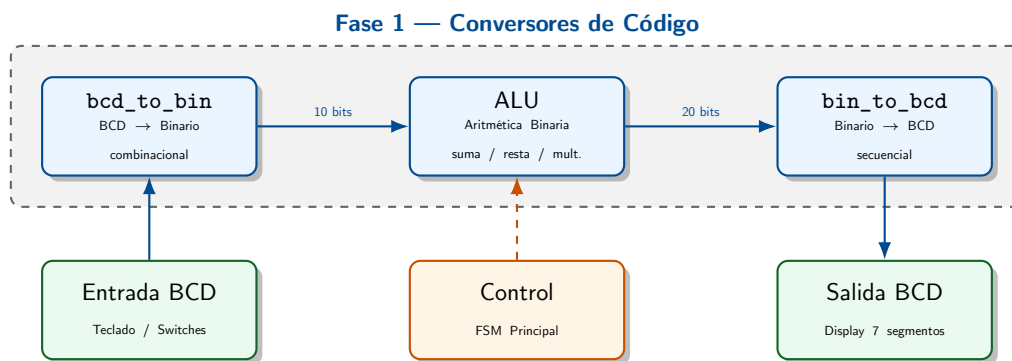
En esta primera fase el trabajo ha consistido en proponer el **diseño jerárquico** de la calculadora y en modelar, simular y depurar los dos conversores de código: **BCD a Binario** (`bcd_to_bin`) y **Binario a BCD** (`bin_to_bcd`).

Estos dos módulos son los más importantes de cara a la interfaz del sistema. El usuario introduce y recibe los datos en BCD (teclado y display de siete segmentos), pero toda la aritmética interna —suma, resta y multiplicación— se hace en binario en complemento a dos. Por eso necesitamos estos conversores: actúan de puente entre los dos dominios.

**Restricción de librería:** Siguiendo las directrices de la asignatura, todos los módulos emplean `ieee.std_logic_unsigned`. Este paquete no soporta la multiplicación directa de vectores de longitud arbitraria, por lo que se recurre a desplazamientos (shifts) y sumas, que son la representación hardware natural de las potencias de dos.

### 1.1. Arquitectura jerárquica del sistema

La calculadora se estructura de forma jerárquica con los siguientes bloques principales:



Los bloques `bcd_to_bin` y `bin_to_bcd` son, por tanto, la frontera entre el dominio decimal del usuario y el binario interno. En esta fase nos hemos centrado en diseñarlos y verificarlos con sus respectivos testbenches.

## 2. Conversor BCD a Binario (`bcd_to_bin`)

### 2.1. Especificación

**Tipo:** Lógica combinacional pura (sin reloj).  
**Entrada:** `bcd(11..0)` — tres nibbles: centenas `bcd[11:8]`, decenas `bcd[7:4]`, unidades `bcd[3:0]`.  
**Salida:** `bin(9..0)` — valor binario en `[0, 999]`.

La conversión es simplemente:

$$\text{bin} = c \times 100 + d \times 10 + u$$

El máximo posible es  $9 \times 100 + 9 \times 10 + 9 = 999$ . Con 10 bits se pueden representar hasta  $2^{10} - 1 = 1023$ , así que no hay riesgo de desbordamiento en ningún caso válido.

### 2.2. Implementación mediante desplazamientos

Como no se puede usar multiplicación directa con `std_logic_unsigned`, hemos descompuesto los factores 100 y 10 como suma de potencias de dos y los hemos implementado con concatenaciones de ceros (equivalente a desplazamientos a la izquierda):

$$c \times 100 = c \times 64 + c \times 32 + c \times 4 = (c \ll 6) + (c \ll 5) + (c \ll 2)$$

$$d \times 10 = d \times 8 + d \times 2 = (d \ll 3) + (d \ll 1)$$

```

1  -- Extender los tres dígitos BCD a 10 bits cada uno
2  c <= "000000" & bcd(11 downto 8);    -- centenas
3  d <= "000000" & bcd( 7 downto 4);    -- decenas
4  u <= "000000" & bcd( 3 downto 0);    -- unidades
5
6  -- c * 100 = c*64 + c*32 + c*4 (64+32+4 = 100)
7  c100 <= (c(3 downto 0) & "000000") -- c * 64 (shift 6)
8  + (c(4 downto 0) & "00000")    -- c * 32 (shift 5)
9  + (c(7 downto 0) & "00");      -- c * 4 (shift 2)
10
11 -- d * 10 = d*8 + d*2 (8+2 = 10)
12 d10 <= (d(6 downto 0) & "000")    -- d * 8 (shift 3)
13 + (d(8 downto 0) & '0');          -- d * 2 (shift 1)
14
15 bin <= c100 + d10 + u;

```

Listado 1: Núcleo de bcd\_to\_bin.vhd — multiplicación por desplazamientos

**⚠️ Anchos de bits:** Cada concatenación produce exactamente 10 bits, igual que la señal destino. Por ejemplo, `c(3 downto 0) & "000000"` produce  $4 + 6 = 10$  bits. Con `std_logic_unsigned` las sumas operan en ese ancho fijo, descartando cualquier carry sin riesgo de desbordamiento silencioso.

## 2.3. Verificación

El testbench `tb_bcd_to_bin` simplemente va cambiando la entrada cada 20 ns sin necesidad de reloj, ya que el módulo es puramente combinacional. Los casos cubren los extremos y algunos valores intermedios:

Cuadro 1: Casos de prueba del testbench `tb_bcd_to_bin`

| Entrada BCD (hex) | Salida binaria (decimal) | Propósito                  |
|-------------------|--------------------------|----------------------------|
| 0x000             | 0                        | Cero absoluto              |
| 0x010             | 10                       | Solo decenas               |
| 0x100             | 100                      | Solo centenas              |
| 0x123             | 123                      | Valor mixto representativo |
| 0x999             | 999                      | Máximo absoluto            |

## 3. Conversor Binario a BCD (`bin_to_bcd`)

### 3.1. Especificación

**Tipo:** Lógica secuencial — FSM de tres estados.  
**Entradas:** `clk` (50 MHz), `nRst` (reset activo bajo), `start`, `bin(N_BITS-1..0)`.  
**Salidas:** `done` (pulso de un ciclo), `bcd(23..0)` — seis dígitos BCD.  
**Genérico:** `N_BITS = 20`  $\Rightarrow$  rango  $[0, 2^{20} - 1] = [0, 1\,048\,575]$ .

La calculadora tiene que poder mostrar resultados de hasta  $999 \times 999 = 998\,001$ , así que el conversor trabaja con 20 bits de entrada y produce 6 dígitos BCD (24 bits). Hacer esto con lógica combinacional pura sería inviable, así que hemos optado por una **arquitectura secuencial iterativa**.

### 3.2. Algoritmo: Double Dabble

Hemos implementado el algoritmo *Double Dabble* (también llamado “shift-and-add-3”), que procesa el número binario bit a bit, de más significativo a menos, en exactamente  $N_{\text{BITS}} = 20$  ciclos de reloj. Cada ciclo hace lo siguiente:

**Paso 1. Add-3:** para cada nibble BCD del acumulador, si su valor es  $\geq 5$ , se le suma 3. Esto garantiza que tras el desplazamiento siguiente el nibble no desborde el dígito decimal.

*Justificación:* si un nibble vale  $n \in [5, 9]$ , tras un shift izquierda valdría  $2n \in [10, 18]$ , desbordando un dígito decimal. Sumando 3 antes ( $n + 3 \in [8, 12]$ ), el shift produce  $2(n + 3) \in [16, 24]$ , cuyo carry transporta correctamente +1 al nibble superior.

**Paso 2. Shift izquierda:** se desplaza un bit a la izquierda el registro combinado  $[\text{acc\_bcd} \mid \text{bin\_reg}]$ , introduciendo el MSB actual de bin\_reg por la derecha de acc\_bcd.

```

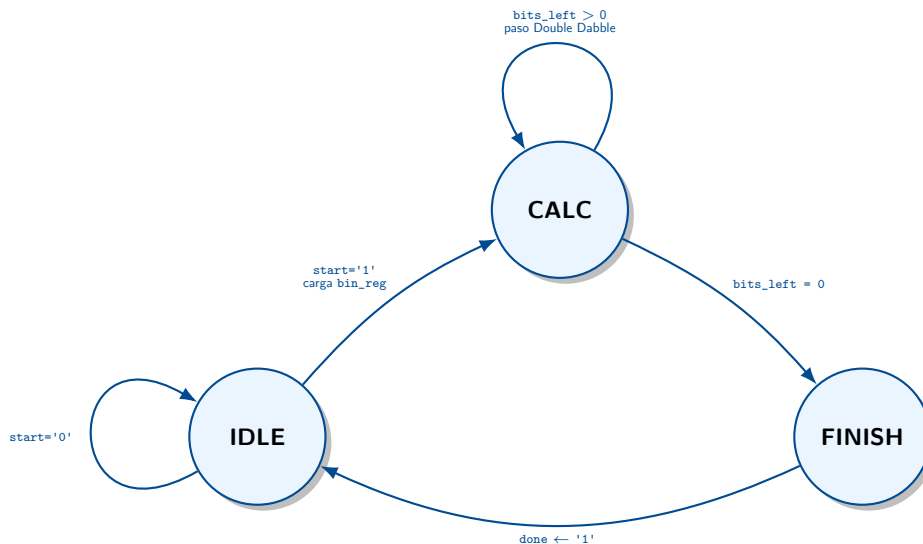
1  -- Paso 1: Add-3 a cada nibble si su valor es >= 5
2  if acc_bcd(3 downto 0) >= "0101" then n0 := acc_bcd(3 downto 0) + "0011";
3  else n0 := acc_bcd(3 downto 0); end if;
4  if acc_bcd(7 downto 4) >= "0101" then n1 := acc_bcd(7 downto 4) + "0011";
5  else n1 := acc_bcd(7 downto 4); end if;
6  -- (idem para n2..n5: centenas, miles, decenas de miles, centenas de miles)
7
8  -- Paso 2: Reconstruir vector ajustado y desplazar
9  bcd_adj := n5 & n4 & n3 & n2 & n1 & n0;
10
11  acc_bcd <= bcd_adj(22 downto 0) & bin_reg(N_BITS-1); -- shift + MSB entrante
12  bin_reg <= bin_reg(N_BITS-2 downto 0) & '0';         -- avanzar registro
13  -- binario
14  bits_left <= bits_left - 1;

```

Listado 2: Núcleo del algoritmo Double Dabble en bin\_to\_bcd.vhd

### 3.3. Máquina de Estados (FSM)

El control secuencial se implementa mediante una FSM de tres estados:



Cuadro 2: Estados de la FSM del conversor bin\_to\_bcd

| Estado | Descripción                                                    | Acción sobre señales          |
|--------|----------------------------------------------------------------|-------------------------------|
| IDLE   | Espera activa de start='1'. Carga bin_reg $\leftarrow$ bin.    | done $\leftarrow$ '0'         |
| CALC   | Ejecuta un paso Double Dabble por ciclo durante N_BITS ciclos. | acc_bcd, bin_reg actualizados |
| FINISH | Pulso de fin; retorna inmediatamente a IDLE.                   | done $\leftarrow$ '1'         |

### 3.4. Verificación

El testbench `tb_bin_to_bcd` genera un reloj de 50 MHz (periodo 20 ns). Nos encontramos con problemas de condiciones de carrera al detectar `done='1'`, así que lo resolvimos con un procedimiento auxiliar (`run_test`) que alinea todo con flancos de reloj. Para simular los diez casos completos en ModelSim hay que ejecutar `run 8000 ns`.

```

1  procedure run_test(val : std_logic_vector(19 downto 0)) is
2      begin
3          wait until rising_edge(clk); -- alinear con el reloj antes de asignar
4          bin <= val;
5          start <= '1';
6          wait until rising_edge(clk); -- IDLE detecta start; done pasa a '0'
7          start <= '0';
8          wait until done = '1';      -- esperar fin de la conversion (20 ciclos)
9          wait for clk_period * 2;    -- margen para visualizar en waveform
10         end procedure;

```

Listado 3: Procedimiento auxiliar `run_test` del testbench secuencial

Cuadro 3: Casos de prueba del testbench `tb_bin_to_bcd`

| Binario (decimal) | BCD esperado (hex) | Propósito                            |
|-------------------|--------------------|--------------------------------------|
| 0                 | 0x000000           | Cero absoluto                        |
| 1                 | 0x000001           | Unidad mínima                        |
| 9                 | 0x000009           | Máximo de un dígito                  |
| 10                | 0x000010           | Decena exacta                        |
| 100               | 0x000100           | Centena exacta                       |
| 255               | 0x000255           | Potencia de 2 menos 1                |
| 999               | 0x000999           | Máximo operando de entrada           |
| 1 000             | 0x001000           | Paso de millar                       |
| 9 999             | 0x009999           | Máximo de cuatro dígitos             |
| 999 999           | 0x999999           | Máximo BCD representable (6 dígitos) |

*Nota: el valor máximo operacional de la calculadora es  $999 \times 999 = 998\,001$ . El último caso de prueba utiliza 999 999 porque es el máximo que los 24 bits BCD de salida pueden representar, ofreciendo así una verificación de frontera más exigente.*

## 4. Conclusiones

En esta primera fase hemos diseñado, implementado y verificado los dos conversores de código que forman la base del proyecto:

- `bcd_to_bin`: completamente combinacional. Hemos resuelto la multiplicación por 100 y por 10 mediante desplazamientos y sumas, sin usar multiplicadores, lo que lo hace compatible con `std_logic_unsigned`. La latencia es nula (solo depende de la propagación de puertas).
- `bin_to_bcd`: secuencial con FSM de tres estados (IDLE / CALC / FINISH). El algoritmo *Double Dabble* tarda exactamente 20 ciclos de reloj, lo que es más que suficiente para los valores que puede generar la calculadora ( $999 \times 999 = 998\,001$  como máximo).

Todos los casos de prueba han dado el resultado esperado, incluyendo el cero, los valores de frontera y algunos intermedios. Los módulos quedan listos para integrarse en las siguientes fases.

→ **Próxima fase:** En la Fase 2 habrá que diseñar la ALU: suma, resta y multiplicación de operandos con signo en complemento a dos (10 bits), con detección de desbordamiento mediante la señal `overflow`.